Build walkthrough

Part	What you build	What proves it works
1A. Sequential console validator	Read and validate an NDJSON file one line at a time.	The five-line fixture produces the exact expected report.
1B. Concurrent console validator	Use one file reader, a bounded record channel, and four consumers.	The report matches the baseline, a full channel slows the reader, and cancellation or failure shuts down the pipeline.
1C. In-memory job API	Add endpoints, job tracking, and a separate bounded queue for whole jobs.	You can submit, inspect, cancel, and retrieve reports. A full job queue returns 503.

1A. Build the sequential console validator

The console app takes a path to an NDJSON file. It reads a line, validates it, updates the report, and then reads the next line. Keep validation of a single line separate from reading and aggregating the file so you can reuse the same rules in 1B.

• Count each line once toward Total. A line is either valid or invalid.

• Count each applicable error category. A line with a blank ID and unsupported currency contributes one invalid record and two error counts.

• Keep the first 20 invalid lines by line number as samples. Each sample contains the line number and all its errors. Keep counting every error after the sample list fills up.

• Treat malformed JSON as a finding for that line and continue. If the file itself can’t be opened or read, the run fails.

Pass when the five-line fixture returns 5 total, 2 valid, 3 invalid, with one occurrence of each category. Also run an empty file and a file with more than 20 invalid lines to check the report boundaries.

1B. Add bounded concurrency inside one validation job

Use one producer to read the file. It sends each line and its original line number into a bounded record channel with capacity 32. Start four consumers. Each takes one record, validates it using the rules from 1A, updates its own counters, and then takes another.

The producer awaits each write before reading the next line. When the channel fills up, the next write waits. A slot opens as soon as a consumer takes a record, even if it’s still validating that record. The 32 slots limit waiting records. The four consumers limit active validations.

After a successful run, combine the consumers’ counters. To preserve the baseline’s sample policy, have each consumer retain its own 20 lowest-numbered invalid lines. At the end, merge those small lists and select the 20 lowest line numbers overall. This keeps the report stable even when line 50 finishes before line 12.

Build walkthrough · continued

Make the pipeline shut down correctly

Event	Expected behavior
Normal end of file	The producer signals that no more records are coming. Consumers finish the records already queued, then exit. The caller combines their results.
Cancellation	The stop signal reaches the producer and all consumers. A producer waiting for channel space can exit. The caller waits for all tasks to finish and treats the run as canceled.
Unexpected producer or consumer failure	The failure signals the other tasks to stop. The caller waits for them to exit and surfaces the original failure.

The important trap: waiting for all tasks to finish doesn't itself tell them to stop. If a consumer fails and the producer is blocked on a full channel, the failure must trigger a stop signal while the tasks are still running. Decide who sends that signal before writing the orchestration.

Prove it with four checks

1. Same result. Run both validators on the same fixtures. Compare totals, category counts, and sampled line numbers.

2. Backpressure. Slow down validation deliberately and observe the producer waiting on a write. Track how many validations are active and verify the peak is at most four. The delay is there to expose the behavior.

3. Cancellation while blocked. Hold the consumers at a point you control, let the buffer fill, and confirm that the producer's next write is pending. Then cancel and verify all tasks exit.

4. Failure while blocked. Recreate the same setup, but make a consumer throw after the write is pending. Verify the caller sees that failure and the producer and remaining consumers exit. A test timeout can catch a hang. Use the controlled pause to arrange the scenario.

1C. Put the validator behind an in-memory job API

Now add a second queue. The queue in 1B holds records from one file. This new queue holds whole jobs waiting to start. Use one hosted background worker that processes one job at a time. For an easy manual test, give the job queue two waiting slots. Each active job still uses the record channel and four consumers from 1B.

Keep an in-memory registry keyed by job ID. It holds the dataset reference, current status, and any completed report. Use a dataset ID that maps to a pre-staged file.

Endpoint	Behavior

Build walkthrough · continued

POST /jobs	Accept a dataset ID. If there's room, register and enqueue the job, then return 202 Accepted with its ID. If the queue is full, return 503 promptly.
GET /jobs/{id}	Return Queued, Running, Succeeded, Failed, or Canceled.
GET /jobs/{id}/report	Return the completed report for a succeeded job. Return 409 for a known job without a successful report and 404 for an unknown ID.
POST /jobs/{id}/cancel	Cancel a queued job so the worker skips it. For a running job, signal the pipeline to stop. Repeated requests are safe, and a terminal state stays terminal.

There are two races worth thinking through. Submission and pickup can overlap, so a worker that dequeues a job must be able to find its registry entry. A rejected submission should leave no job behind that looks accepted. Cancellation and completion can also overlap, so make the final transition atomic. If success wins, the status and report should become visible together. If cancellation wins, the worker shouldn't later publish a success report.

Run the final milestone demo

1. Submit job A and hold it in Running. Submit jobs B and C. They should be Queued. Submit job D. It should receive 503.

2. Cancel one queued job and verify the worker skips it. Let another accepted job finish and retrieve its report using the original job ID.

3. Cancel a running job and verify its pipeline exits and its state becomes Canceled.

4. Make one job fail unexpectedly. Verify it becomes Failed and the hosted worker can still pick up the next job.

That's the full scope of Milestone 1. Its job state lives in memory for the lifetime of the application. Milestone 2 starts when you add PostgreSQL to make accepted jobs survive a restart.

Checkpoints 01–02

1A. Sequential console validator

Checkpoint 1 — Match the five-line report by hand

Set up. Use the five-line orders.ndjson fixture. Its expected result is independent of your implementation.

Line	Expected result
1	Valid
2	InvalidAmount
3	InvalidId and UnsupportedCurrency
4	MalformedJson
5	Valid

Run. Pass the file path to your console app and compare the report against the table.

Pass when. The report shows 5 total, 2 valid, 3 invalid, one occurrence of each error category, and samples for lines 2, 3, and 4 in that order. Line 3 has both errors in the same sample. Line 5 contributes to the valid count, which proves the validator continued after malformed JSON.

Checkpoint 2 — Verify the boundaries

Run these separately so a failure tells you which behavior to inspect.

Input	Expected result
Empty file	Zero records, zero errors, and no samples.
25 invalid lines, each with a negative amount and otherwise valid fields	25 total, 25 invalid, 25 InvalidAmount findings, and samples for lines 1–20 only.
File path that doesn't exist	The run fails with a file error. The app surfaces the failure instead of printing a successful validation report.
Large generated file	The reader processes the file incrementally. The validator should keep aggregate counters and capped samples rather than accumulating every parsed record or error.

Before moving on. Make sure you can reuse the rules for validating one line independently of the code that reads and aggregates the whole file. Keep the sequential implementation so you can compare it against the concurrent one.

Checkpoints 03–06

1B. Concurrent console validator

Checkpoint 3 — Get the same report with four consumers

Set up. Use one file reader, a record channel with 32 waiting slots, and four consumers. Give each record its original line number before enqueueing it. Each consumer validates one record at a time and keeps its own counts and samples.

Run. Compare the concurrent report against the sequential report for the five-line fixture and a larger seeded fixture with more than 20 invalid lines. For the larger fixture, introduce different validation delays across records in a test so completion order can differ from file order.

Pass when. Both implementations agree on totals, category counts, and the same first 20 invalid line numbers, including the errors on each sampled line. Compare the report data rather than console whitespace or execution time.

Hint for samples. Each consumer keeps its 20 lowest-numbered invalid lines. Merge those candidates and take the 20 lowest overall. If a consumer discards an invalid line, it already has 20 earlier invalid lines of its own, so the discarded line could never belong in the global first 20.

Checkpoint 4 — Observe backpressure and the concurrency limit

Set up. Use at least 100 records. Give consumers a test-only pause after they take a record. Track how many validations are active and whether a producer write has to wait.

Run. Let all four consumers take a record and pause them. Allow the producer to fill all 32 waiting slots. Its next write should remain pending. Release one consumer and let it finish its current record and take another.

Pass when. Peak active validations is at most four. The pending write completes after the consumer takes another record and frees a channel slot. Let the run finish and confirm the report includes every record. This proves the producer waited for capacity and resumed without dropping data.

Checkpoint 5 — Cancel while the producer is waiting

Set up. Recreate the full-channel situation. Make the consumers' test pause respond to cancellation too. Wait for a signal that the producer's write is pending before triggering cancellation.

Run. Cancel the validation run while the write is still pending.

Pass when. The pending write exits, all four consumers exit, and the caller observes cancellation. The caller does not receive a successful partial report. Use a timeout to catch a hang. The pending-write signal is what tells you when to cancel.

Checkpoint 6 — Fail a consumer while the producer is waiting

Set up. Use the same full-channel setup. Arrange for one consumer to throw a deliberate exception when you release its test gate. Keep the other consumers paused at waits that respond to the stop signal.

Checkpoints 03–06 · continued

Run. Wait for the producer’s write to be pending, then trigger the exception in that one consumer.

Pass when. The producer and the other consumers exit, and the caller sees the injected exception as the cause of failure. A sibling task may exit through cancellation during cleanup, but the run’s outcome is failed.

Prediction to write first. Who notices the consumer’s failure and signals the producer to stop? If your only plan is to handle the exception after waiting for every task to finish, trace what happens to the producer that’s still waiting for space.

Checkpoints 07–10

1C. In-memory job API

Use one active job and two waiting job slots for these checks. Those are limits on whole files. The active job still has its own record channel and four consumers. Give yourself a test gate that can hold a job in Running until you release it.

Checkpoint 7 — Submit a job and get its report later

Run. Submit a known dataset. Hold the worker after it claims the job and use the returned job ID to query status and request the report. Then release the worker and query both again.

Pass when. Submission returns 202 with a job ID while validation is still running. During the pause, status is Running and the report endpoint returns 409. After release, status is Succeeded and the report matches the console baseline. If a client sees Succeeded, it can retrieve the report. An unknown ID returns 404.

Checkpoint 8 — Fill the job queue and reject excess work

Run. Hold job A in Running. Submit B and C to fill the two waiting slots. Submit D while A is still paused.

Pass when. A, B, and C all received 202 and remain queryable. B and C are Queued. D receives 503 promptly and leaves no accepted job in the registry. Release A and check that the worker processes the queued jobs. This checks API admission. The record-level test in checkpoint 4 checked backpressure inside a job.

Checkpoint 9 — Cancel queued and running work

Scenario	How to arrange it	Pass when
Queued cancellation	Hold A in Running, submit B, then cancel B before releasing A.	B becomes Canceled. The worker skips validation and produces no report. B can still occupy a queue slot until dequeued.
Running cancellation	Hold A inside validation at a cancellable wait, then request cancellation.	The pipeline exits, A becomes Canceled, no success report appears, and the worker can run another job.
Cancellation races completion	Pause A before publishing success. In one run, let cancellation win. In another, let success commit before sending cancellation.	A ends Canceled with no report in the first run. A stays Succeeded with its original report in the second. Repeating cancellation preserves the terminal outcome.

Checkpoint 10 — Fail one job and keep the worker alive

Run. Submit A and B. Inject an unexpected validation failure into A. Use a normal dataset for B.

Checkpoints 07–10 · continued

Pass when. A becomes Failed and has no successful report. B still runs and becomes Succeeded. This verifies that one job’s pipeline can fail without taking down the hosted worker that handles the queue.

Milestone 1 is done when all ten checks pass. If you’re still on the five-line fixture, your next two checks are the exact baseline report and the 20-sample cap. Once those pass, checkpoint 3 is where you introduce the channel.